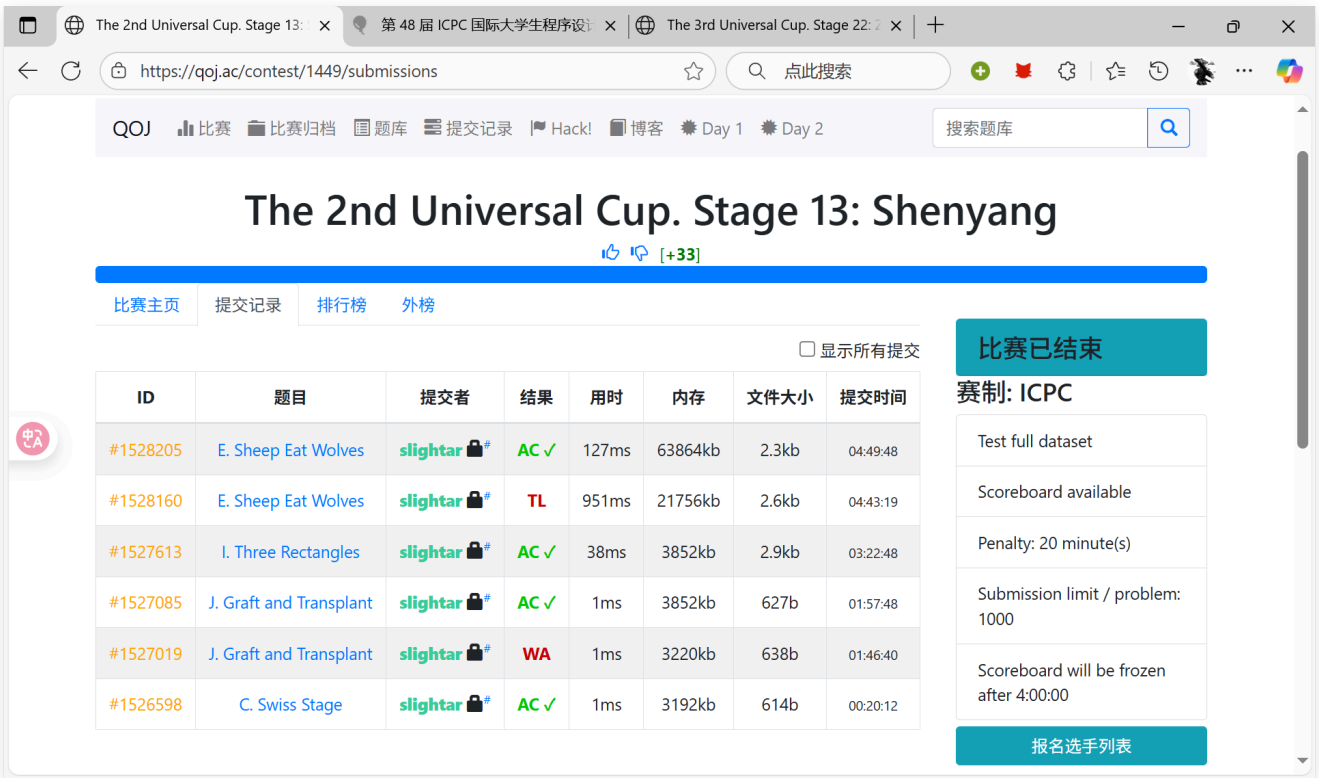
0. VP情况

先贴上三个链接, vp/补题、官方题解、赛时榜单



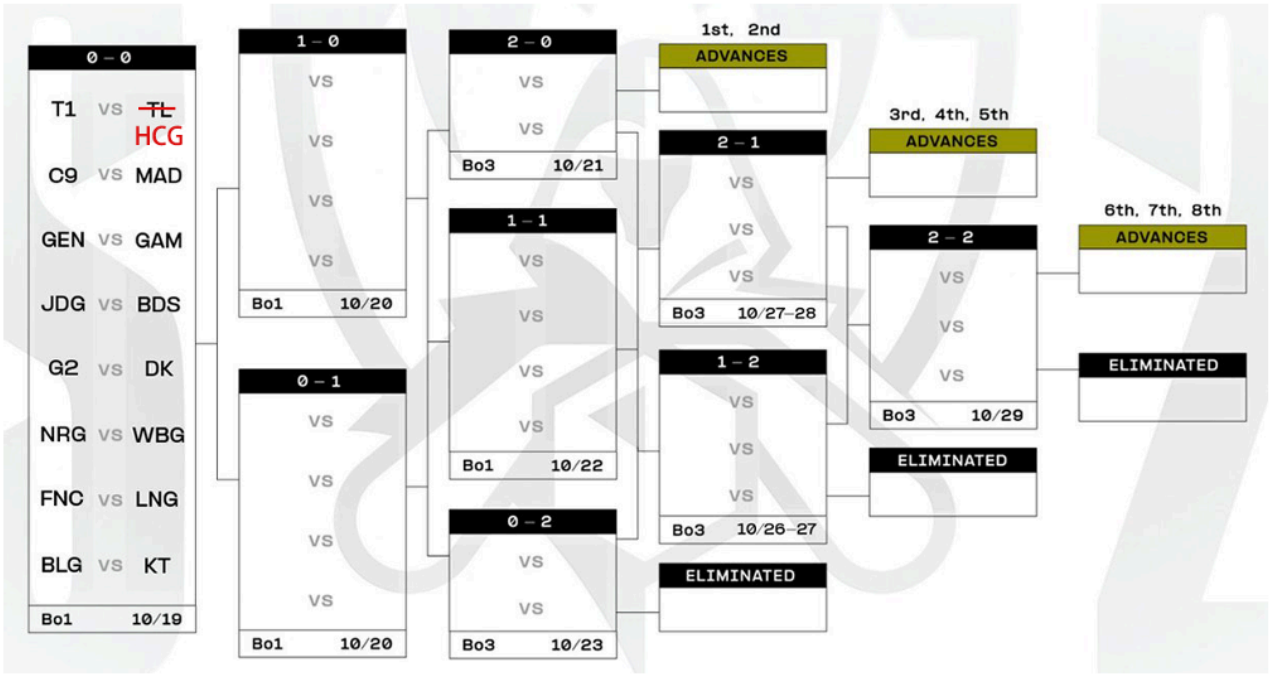
赛站	铜牌线	银牌线	金牌线	过题	vp 成绩	vp 日期	备注
IC 济南	4-806	5-741	7-774				
IC 合肥	3-435	5-1019	6-728				
IC 澳门	2-332	4-562	6-718				
IC 沈阳	3-304	4-345	6-871		4-668	QOJ 10-16	I题分讨金牌题，难以置信
IC 南京	3-391	5-417	7-961				
IC 西安	3-712	4-984	5-738				
CC 成都(final)	4-665	5-848	6-896				

赛时过了四道题, 结合罚时大概是铜牌平均水平, 过题顺序分别是$C、J、I、E$, 值得一提的是I题在赛时仅仅过了二十多支队伍, 是金牌题 以下按照个人感觉的题目难度给出题解, 分别为$C、J、E$

C. Swiss Stage

1. 题面简述

现在是一场LOL赛事, 有16支队伍即将进行5轮比赛, 赢得其中3轮的队伍可以晋级, 每一轮的比赛输赢规则并不一样, 有的轮是 **"best-of-three"(Bo3)** 规则, 另一些是 **"best-of-one"(Bo1)规则**, 具体的如图所示：



The Swiss Stage of Worlds 2023
(Credit: Riot Games)

现在告诉你某一只队伍的已经赢了的轮次 x 和输了的轮次 y (均小于3), 请给出这只队伍想要晋级最少需要赢得的**场次**.

2. 分析

根据图片打表即可, 只有 $3*3=9$ 种情况, 当然如果想写的更优雅, 也可以去找一个通解, 这里不再赘述。

3. 代码

▶ 点击查看代码

J. Graft and Transplant

1. 题面简述

Alice 和 Bob 正在对一颗树玩游戏, 他们可以进行如下操作:

1. 选中一条边
2. 选择这条边两个顶点中的一个(记作 u ,另一个记作 v)
3. 对于 u 的所有关联顶点(不包括 v), 全部与 u 断联并且连接 v

显然这个操作没有什么限制, 只要存在边就可以一直进行下去, 所以他们对于这个游戏加上了一个限制**每次操作后的树不可以与操作前的树同构**, 并且题目中给出同构的定义:

> Tree S and tree T are isomorphic if there exists a bijection f : $V (S)->V(T)$ such that for all pairs of vertices (u,v) in $V(S)$, u and v are connected by an edge in S if and only if vertices $f(u)$ and $f(v)$ are connected by an edge in T. Namely, $f(s) = t$ implies vertex s in S corresponds to vertex t in T.

现在问你给定一颗树, Alice先手的情况下, 谁会赢得游戏(即无法继续进行限制下的操作)

2. 分析

选择一个悬挂边(含有叶子结点的边), 这样的操作没有任何意义, 并且每次有效操作后, u结点会变成叶子结点, 而其他节点的度数不会改变, 所以每次操作叶子个数会加一, 所以能进行操作的次数是非叶子结点的个数, 奇偶性就决定了谁会赢, 当然, 如果一个叶子结点都没有, Alice直接就输掉了比赛, 所以对于0的情况需要特判一下。

3. 代码

▶ 点击查看代码

E. Sheep Eat Wolves

1. 题面简述

你是一个牧羊人, 现在一条河阻挡了你带小羊回家的路, 河中有一条船, 船的承载能力是有限的, 并且有一群狼在虎视眈眈. 现在给出羊的数量 x, 狼的数量 y, 船的承载数量 p (最大能载动物的数量,人不计入), 狼的凶猛程度 q, 狼的凶猛程度指的是只有当 $yy-xx>q$ (xx,yy 是羊和狼此刻的数量)并且牧羊人不在场时, 狼就会吃羊. $ x,y,z,q<100 $ 现在请问牧羊人摆渡几次能在羊不被吃的情况下带所有的羊过河, 如果无法做到, 输出-1.为了帮助理解, 这里给出样例: 输入:

```
3 5 2 0
```

预期输出:

```
5
```

因为当狼羊数量差大于0, 羊就会被吃, 所以我们的第一次渡河必须带两只狼过去, 然后空船回来, 此时带一羊一狼过去, 但是这样河对岸就会有三狼一羊, 然后返程时必须带两只狼回来, 最后将两只羊带过去, 任务完成, 总共渡船五次.

2. 分析

注意到数据范围很低, 只考虑河的一边的话, 所有的可能性最多只有 $100^4=1e8$ 种状态, 所以我们可以使用BFS进行暴力搜索, 找出步数最小的送羊方案.

3. 代码

▶ 点击查看代码